

SAPTHAGIRI NPS UNIVERSITY

[Department Name]

CEREBROKOGNIT

Intelligent Cognitive Alarm Platform

A project report submitted in partial fulfilment of the requirements for the degree of Master of Computer Applications (MCA), as part of the Infosys Virtual Internship 7.0 programme.

Presented by

Pawan Simha R

[Registration Number / USN]

Presented to

Shanmukha Priya

Project Guide and Mentor

Academic Year: [Academic Year]

Table of Contents

1. Introduction 3
1.1 Project Background 3
1.2 Problem Statement 3
1.3 Objectives 3
1.4 Scope 3
2. Methodology 4
2.1 Methodology Overview 4
2.2 System Architecture 4
2.3 Functional Methodology 5
3. System Design 6
3.1 System Workflow 6
3.2 System Components 6
4. Implementation 8
4.1 Development Environment and Project Structure 8
4.2 Core Module Implementation 9
4.3 Database Implementation and Data Processing 9
4.4 Navigation Workflow and Validation Mechanism 10
4.5 User Interface Implementation 12
5. Testing and Results 15
5.1 Testing Approach 15
5.2 Test Cases 15
5.3 Results and Observations 15
6. Conclusion and Future Enhancements 16
6.1 Conclusion 16
6.2 Future Enhancements 16
7. Appendix 17
Appendix A: Software and Hardware Requirements 17
Appendix B: Project Structure 17
Appendix C: Database Information 17
Appendix D: Application Modules 17
Appendix E: Source Code Repository 17

1. Introduction

1.1 Project Background

Waking up on time is a persistent behavioural challenge. Conventional digital alarms can be dismissed within seconds, before the sleeper is fully alert, and repeated snoozing fragments the final sleep cycle of the night. CerebroKognit — the Intelligent Cognitive Alarm Platform — gates alarm dismissal behind personalised cognitive challenges: an alarm is cleared only after the user correctly solves questions drawn from seven challenge types (mathematics, logic, memory, word, pattern, riddle, and quick quiz). Developed under the Infosys Virtual Internship 7.0, it is a three-tier system: a FastAPI backend, a React web dashboard, and an Expo mobile application with offline-first synchronisation.

1.2 Problem Statement

Existing alarm applications verify only that a button was pressed. Users with chronic oversleeping tendencies dismiss alarms reflexively and return to sleep. Ad-hoc solutions such as QR-code or photo tasks are static: they prove physical movement but impose no mental engagement and do not adapt to the user. A more effective alarm must require verifiable cognitive effort and calibrate difficulty to the individual.

1.3 Objectives

Provide time-zone-aware alarm scheduling with recurrence, snooze tracking, and question-count configuration.

Enforce a server-authoritative challenge gate in which correct answers are never exposed to the client.

Adapt difficulty using a trained model, rule-based signals, and a contextual bandit, and compute daily habit scores, analytics, and reports.

Deliver role-based access with secure authentication and audit logging; support offline-first mobile use and Docker deployment with automated tests.

1.4 Scope

The release includes the FastAPI interface (69 routes under /api/v1 across 12 routers), 14 SQLAlchemy models persisted to PostgreSQL or SQLite, the React dashboard, the Expo mobile application, local notification scheduling, CSV/JSON/PDF/Excel exports, and pytest, Vitest, and Jest suites in continuous integration. It excludes cloud deployment, mandatory Redis or MongoDB services, remote push on Android without an Expo development build, and models trained on real user data.

2. Methodology

2.1 Methodology Overview

Development followed a milestone-driven agile process over eight weeks. Milestone 1 (Foundation) established project scaffolding, the relational schema, JWT authentication with role-based access control, user profiles, and time-zone-aware alarm scheduling. Milestone 2 (Cognitive Engine) built the server-authoritative challenge service, puzzle generation from seeded randomness, the submit-and-dismiss verification flow, and challenge analytics. Milestone 3 (Adaptive Intelligence) added habit scoring, the analytics service, the recommendation engine, and the machine-learning inference path with a rule-based fallback. Milestone 4 (Polish and Deploy) delivered the dashboards, PDF and Excel reports, an error envelope, continuous integration, Docker deployment, and the first regression suites. Later hardening phases added refresh-token rotation and audit logging, mobile push registration, on-device local scheduling, dead-code cleanup, and a product-wide visual redesign.

Two design decisions shaped the whole system. First, every dismissal decision is made server-side: the challenge payload sent to a client contains questions but never expected answers, and a dismissal is accepted only after the server validates a submitted attempt against stored answers within an expiry window. Second, the system degrades gracefully — SQLite self-seeds roles, demo accounts, and a 7×5 challenge matrix on first start, and machine learning falls back to a transparent rule engine when the trained artifact is unavailable, so the platform remains fully operable in minimal environments.

2.2 System Architecture

The architecture is a three-tier client/server design (Figure 2.1). The presentation tier comprises the React 18 single-page dashboard, served by Vite in development and nginx in the production Docker overlay, and the Expo mobile application. Both communicate with the application tier — a FastAPI service organised into routers, domain services, an AI engine package, and a gateway layer that applies cross-origin policy, rate limiting, security headers, and structured request logging. The data tier is PostgreSQL 15 in Docker for shared deployments and SQLite for local development, accessed through SQLAlchemy 2.0 and versioned with Alembic migrations. Optional integrations (an event log and error monitoring) attach at the service layer without being required for core operation.

2.3 Functional Methodology

Functionally, the platform processes user requests through a uniform pipeline: middleware assigns request identifiers and enforces an in-memory sliding-window limit of 60 requests per minute; router dependencies resolve the bearer token to a current user and enforce role requirements; Pydantic v2 schemas validate the payload; a domain service executes the operation against the ORM; and errors are returned in a single structured envelope with detail, error code, timestamp, and status. The core cognitive loop works as follows: when an alarm fires, the client requests the next challenge; the server generates questions and expected answers from one seed, stores the attempt with an expiry, and returns only the questions; the user's submission is compared server-side; a successful attempt permits the dismiss call, which records the wake event and recomputes the habit score. An adaptive loop then extracts behavioural features from the attempt to choose the next difficulty level using the trained model, the rule thresholds, or the UCB1 bandit, whichever path the deployment has enabled.

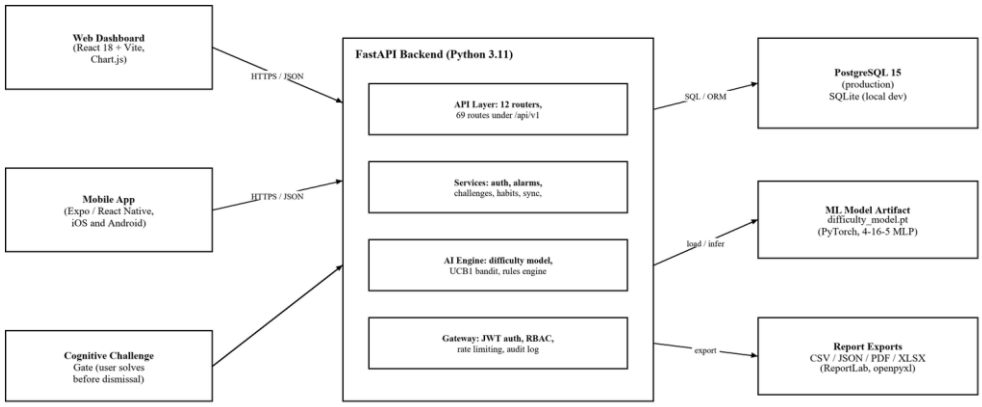


Figure 2.1: System architecture of the Intelligent Cognitive Alarm Platform

3. System Design

3.1 System Workflow

The workflow begins when a user creates an alarm: the client submits a title, alarm type, question count, and preferred challenge, and the backend computes the next trigger instant in the user's stored time zone, honouring recurrence rules. At trigger time the mobile application raises a local notification on the dedicated cerebrokognit-alarms channel, or the web dashboard surfaces the pending alarm. Opening the alarm routes the user to the challenge gate, which fetches the next challenge over GET /challenges/next; the response carries puzzle content only. The user answers N questions before the attempt expires and submits them to POST /challenges/submit, where the server checks each answer against stored expected answers, marks the attempt consumed so it cannot be replayed, and returns a verdict. A pass allows POST /alarms/{id}/dismiss, which closes the alarm and writes the day's habit-score components; a fail or expiry records an incomplete attempt and leaves snooze or retry as the remaining options. Offline, the mobile client queues actions locally and later replays them through POST /sync/pending as idempotent upserts, then merges server state from GET /sync/pull.

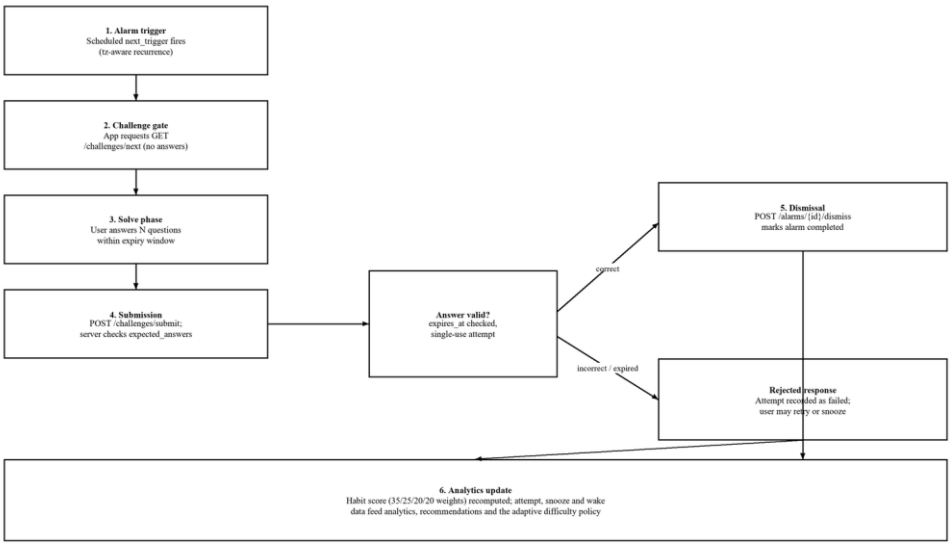


Figure 3.1: Alarm-dismissal and challenge-verification workflow

3.2 System Components

The system decomposes into the components listed in Table 3.1. The API layer exposes twelve routers — authentication, users, alarms, challenges, habits, sync, dashboard, analytics,

notifications, feedback, productivity, and announcements — under /api/v1. Domain services encapsulate authentication and token rotation, puzzle generation, dismissal verification, habit scoring, analytics, recommendations, notifications, reporting, and administration; the AI engine package holds the difficulty predictor, feature rules, the contextual bandit, anomaly checks, and the performance forecast. Clients communicate exclusively over JSON REST calls with bearer tokens: the web dev server proxies /api to the backend, and the mobile client derives the backend host at runtime from the Metro bundle address.

Component	Technology	Responsibility
API service	FastAPI (Python 3.11)	69 REST routes, auth, validation, RBAC
Domain services	Python modules	Challenges, alarms, habits, sync, reports
AI engine	PyTorch, pure Python	Difficulty prediction, rules, bandit
Data tier	SQLAlchemy 2.0 / Alembic	14 models, migrations, indexes
Web dashboard	React 18, Vite, Chart.js	User, coach, and admin interfaces
Mobile app	Expo SDK 58, React Native	Alarms, challenge gate, offline sync

Table 3.1: Major system components

4. Implementation

4.1 Development Environment and Project Structure

The backend targets Python 3.11 (the continuous-integration and container baseline) and runs on newer interpreters as well; it is built on FastAPI 0.141 with Starlette, SQLAlchemy 2.0, Alembic 1.13+, PyJWT with HS256, and bcrypt for password hashing. Machine-learning dependencies include PyTorch and a scikit-learn-compatible training script, although the runtime container deliberately omits the heavy libraries and relies on the rule-based path. Reports are produced with ReportLab (PDF) and openpyxl (Excel). The web client uses Node.js 20, React 18, Vite 5, react-router-dom 6, and Chart.js 4 with Vitest for testing; the mobile client uses Expo SDK 58, React Native 0.88, React Navigation 7, expo-notifications, AsyncStorage, and Jest with jest-expo. Development was performed in Visual Studio Code with the Python, FastAPI, Expo, and PostgreSQL extensions; services are launched from single-command launchers (dev.ps1, run_all.ps1, CerebroKognit.bat) that pin the virtual-environment interpreter and wait for port binding. Table 4.1 summarises the toolchain.

Layer	Tools and versions
Backend	Python 3.11, FastAPI 0.141, SQLAlchemy 2.0, Alembic, PyJWT, bcrypt
Machine learning	PyTorch (MLP artifact), rule engine, UCB1 bandit (pure Python)
Data	PostgreSQL 15 (Docker), SQLite (local), Alembic migrations
Web	Node 20, React 18, Vite 5, Chart.js 4, Vitest
Mobile	Expo SDK 58, React Native 0.88, expo-notifications, Jest
Quality / deploy	pytest, Ruff, GitHub Actions, Docker Compose, nginx

Table 4.1: Development environment and toolchain

The repository is a monorepo. The backend application package contains the FastAPI entry point, ORM models, configuration, twelve API routers, domain services, schemas, the AI engine, gateway middleware, and integration adapters; alongside it sit the trained model artifact, operational scripts (database initialisation, smoke tests, model training and evaluation), and the pytest suite. The web and mobile directories each hold a full client application with its own package manifest, test configuration, and assets. Root-level files provide Alembic configuration, Docker Compose files for development and production, continuous-integration workflow definitions, and the launcher scripts. A compact tree appears in Appendix B.

4.2 Core Module Implementation

The authentication service registers users with bcrypt-hashed passwords, issues 60-minute access tokens and 30-day refresh tokens, and stores refresh, password-reset, and email-verification tokens as SHA-256 hashes that are single-use and revocable. Google OpenID Connect sign-in is available when client credentials are configured and returns a clear unavailable response otherwise. The alarm service computes time-zone-aware next-trigger instants, handles snooze counters, and writes habit entries on dismissal. The challenge service generates each puzzle and its expected answers from a single seed, and the verification service enforces the dismissal gate: attempts carry an expiry timestamp and a consumed marker so a solved challenge cannot unlock a second alarm. The habit-scoring service is the single source of truth for the 35/25/20/20 weighting of wake consistency, challenge completion, snooze reduction, and sleep-schedule adherence. The AI engine predicts a recommended difficulty with a 4→16→5 fully connected network (difficulty_model.pt, trained on a synthetic seed-42 data set), blends it with six rule signals and a UCB1 contextual bandit, and contributes anomaly detection and a linear performance forecast to the analytics endpoints. Reporting services render habit, wake-up, sleep, challenge, and productivity reports as CSV, JSON, PDF, or Excel. On the client side, the web application implements landing, authentication, home, dashboard, alarm management, challenge, reports, profile, settings, and admin pages behind a role-filtered five-section navigation; the mobile application implements a four-tab shell with stack screens for login, alarm management, the challenge gate, profile, settings, and feedback, plus an offline store, a synchronisation engine, and notification scheduling helpers.

4.3 Database Implementation and Data Processing

Persistence uses SQLAlchemy 2.0 declarative models mapped to PostgreSQL 15 in Docker or SQLite for development, with the engine selected from the DATABASE_URL configuration. Fourteen tables implement the domain: roles, users, user_roles, alarms, challenges, challenge_attempts, habit_scores, push_tokens, feedback, coach_assignments, auth_tokens, productivity_logs, audit_logs, and announcements. Users carry a unique lower-cased email index, an optional OAuth identity pair, a stored time zone, and a JSON settings blob. Challenge attempts are the security-sensitive core: they separate the client-visible puzzle payload from the server-only expected answers and answer metadata, and add expiry and consumption markers. Habit scores are keyed by user and date string with the four weighted components and an overall score. Foreign-key cascades keep coach assignments,

tokens, attempts, and habit rows consistent when a user is deleted. Schema changes are versioned through eleven Alembic revisions, the current head adding productivity logs, and the Docker entry point applies pending migrations before the API starts. Figure 4.1 shows the principal entities and relationships.

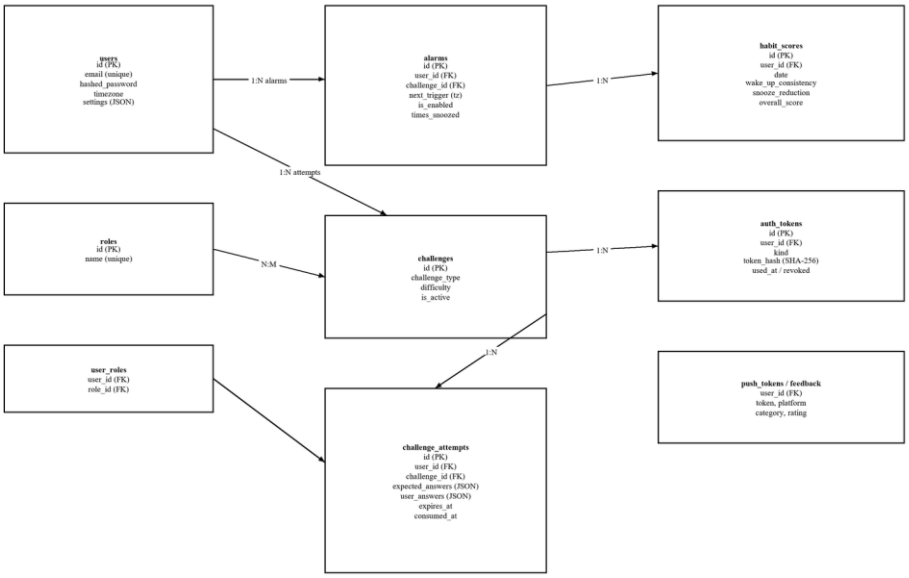


Figure 4.1: Core entity-relationship diagram (principal entities)

Data processing follows the same single-source principle as the rest of the backend. Puzzle content, expected answers, and grading metadata for an attempt are serialised into JSON columns so that the server can re-verify a submission without regenerating it, while the client-visible payload is returned as a filtered projection. Habit scores are written once per user per date, with the analytics service later aggregating them into sleep estimates, wake trends, and recommendations. Report generation reads the same ORM session and streams CSV, PDF, or Excel output through the export services. Migration state is tracked by Alembic, and the local SQLite database self-initialises on startup when no DATABASE_URL is configured, seeding roles, three role accounts, and a 7×5 challenge matrix so a fresh checkout is immediately testable.

4.4 Navigation Workflow and Validation Mechanism

Web navigation is organised into five role-filtered sections — Overview (dashboard, analytics), Wake (alarms, challenges), Insights (history), Account (profile, settings), and Administration (admin console) — rendered as a collapsible sidebar with a global theme toggle; the Administration section and its API routes are visible only to administrators.

Mobile navigation uses a custom bottom tab bar (home, alarms, insights, profile) over a native stack; notification taps deep-link into the alarm's challenge gate. Validation is layered: Pydantic v2 schemas enforce well-formed emails, minimum password length, an allow-list of self-service roles, and enumerated report formats; request dependencies enforce bearer-token authentication and role membership, returning 403 with an explicit role requirement; and the challenge flow enforces temporal validity through attempt expiry and single-use consumption. All failures, including validation errors, are normalised into one structured error envelope, while the gateway applies CORS allow-listing, security headers, request identifiers, and rate limiting, and the audit service records sign-ins, role changes, and password resets.

Failure handling is centralised. Every HTTP error and validation failure is rendered as a single envelope carrying a human-readable detail, a stable error code, a timestamp, and the status code, so clients can branch on the code rather than parse messages. The gateway middleware attaches a request identifier to every call and logs it together with timing information, which allows a failing request to be traced across the API and service layers. In production, the application refuses to start when the signing secret is missing or left at its placeholder value or when the CORS allow-list is empty, converting a silent misconfiguration into an explicit startup failure. A middleware layer also emits standard security headers, and the optional error-monitoring integration reports exceptions with personally identifiable information disabled.

4.5 User Interface Implementation

The user-facing layer was redesigned product-wide into a retro-futuristic visual system: a flat brand palette without gradients or glow, a landing page with live simulator components, grouped navigation, a command-centre home dashboard, and responsive layouts that reflow from four columns to one. Figures 4.2 to 4.7 present interface visuals drawn from the web client's own presentation gallery shipped in the repository; the mobile application follows the same design language with its custom tab bar and screen system.



Figure 4.2: Web application interface visual (landing gallery asset)

The web client is a single-page React application. Unauthenticated visitors see the landing experience; authenticated users are routed to the dashboard shell, which combines live clock and challenge simulator components with data fetched from the analytics, alarm, and habit endpoints.



Figure 4.3: Web application interface visual (landing gallery asset)

Alarm management is a dedicated page backed by the alarms router: users create, edit, enable, and disable alarms, select challenge type and question count, and inspect upcoming

triggers computed in their stored time zone. Snooze and dismiss actions call the corresponding API endpoints directly, mirroring the workflow of Section 3.1.



Figure 4.4: Web application interface visual (landing gallery asset)

Insights and reporting surfaces render habit-score history, wake trends, and challenge statistics with Chart.js, consuming the same analytics endpoints the mobile application uses; habit, wake-up, sleep, challenge, and productivity reports can be exported as CSV, JSON, PDF, or Excel files.



Figure 4.5: Web application interface visual (landing gallery asset)

The administrator console — restricted to the Administrator role — manages user accounts and role assignments, coach-to-user cohorts, and announcements, with every privileged action written to the audit log.



Figure 4.6: Web application interface visual (landing gallery asset)

The mobile application reproduces the core flow on iOS and Android through Expo: alarms are scheduled locally with expo-notifications, notification taps route into the challenge gate, and offline actions queue in the local store and reconcile through the sync endpoints on reconnection.



Figure 4.7: Web application interface visual (landing gallery asset)

All interface visuals are genuine assets of the project's web client; automated tests cover the component layer, while end-to-end on-device notification firing is tracked by a manual real-device checklist, as noted in Section 5.

5. Testing and Results

5.1 Testing Approach

Testing is organised in three suites plus continuous integration. The backend uses pytest against the FastAPI application with an in-process test client and an isolated SQLite database, covering authentication, security hardening, alarms, challenges, analytics, feedback, notifications, synchronisation, and end-to-end workflows; two in-process smoke scripts provide milestone regression gates. The web client uses Vitest with React Testing Library, and the mobile client uses Jest with jest-expo. GitHub Actions runs Ruff linting, the backend suites and smokes on Python 3.11, the web tests and production build, and the mobile Jest suite on every push to main and every pull request. Figures below reflect the repository's test audit of 17 October 2026; earlier documents quote lower counts from previous phases.

5.2 Test Cases

ID	Module	Scenario	Expected result	Actual result	Status
TC-01	Auth	Register, login, refresh	JWT pair; refresh rotates	Verified by auth and security suites	Pass
TC-02	Alarms	Create alarm, cross-TZ	Trigger computed in user TZ	Verified by backend alarm tests	Pass
TC-03	Challenges	Fetch /next, submit	No answers leaked; server verdict	Verified incl. no-leak hardening	Pass
TC-04	Dismissal	Replay consumed attempt	Rejected; single-use enforced	Verified by hardening tests	Pass
TC-05	Habits	Score a day post-dismiss	35/25/20/20 weighted score	Verified by analytics / e2e tests	Pass
TC-06	Sync	Replay offline queue twice	Idempotent, no duplicates	Verified by backend sync tests	Pass
TC-07	Mobile	Fire alarm on device	Local notification opens gate	On-device check not yet executed	Pending

Table 5.1: Representative test cases

5.3 Results and Observations

The audited results of 17 October 2026 record 219 backend tests passing in 76.5 seconds with 87 percent statement coverage over 3,595 statements, a clean Ruff lint run, 22 of 22 milestone-1 smoke checks passing, 58 passing web tests, and 84 mobile tests; continuous integration reproduces these gates on every change. Known limitations are documented rather than hidden: on-device notification delivery is covered only by a manual checklist; remote push on Android requires an Expo development build; and the difficulty model is trained exclusively on synthetic data. Project documents from different phases quote differing totals (187 to 225); the audit above is the most recent verified count.

6. Conclusion and Future Enhancements

6.1 Conclusion

CerebroKognit delivers a working intelligent alarm platform in which dismissing an alarm requires solving server-verified cognitive challenges. The implemented system provides time-zone-aware scheduling, a seven-type, five-difficulty challenge library, a server-authoritative verification gate with expiry and single-use guarantees, adaptive difficulty selection, habit scoring and analytics, role-based access for users, coaches, and administrators, offline-first mobile synchronisation, and multi-format report exports. It is delivered as a documented monorepo with 69 API routes, 14 database models under Alembic versioning, a React dashboard, an Expo mobile application, and a test base of 219 backend, 58 web, and 84 mobile automated tests executed in continuous integration, with Docker-based deployment verified locally. The original objectives — a verifiable cognitive dismissal gate, adaptive difficulty, habit feedback, and cross-platform access — are met in the current release, with the stated limitations around on-device verification and real-data model training.

6.2 Future Enhancements

Complete physical-device verification of notification delivery using the existing real-device checklist, then enable Expo development builds so remote push works on Android. Benefit: guaranteed on-device alarm firing across platforms.

Deploy to a managed cloud environment using the production Compose overlay, including TLS termination and secrets management. Benefit: always-available access beyond local Docker.

Retrain the difficulty model on anonymised real wake and attempt data collected through the existing analytics pipeline. Benefit: personalised difficulty grounded in field behaviour rather than synthetic rules.

Replace the in-memory rate limiter with a shared Redis-backed implementation and promote the optional event log to a durable telemetry store. Benefit: correct limits and audit trails across multiple API instances.

Extend coach tooling with guided interventions and exportable cohort reports. Benefit: stronger wellness-platform use cases already identified as a target persona.

Each of these items is proposed future work; none is represented as implemented in the current release.

7. Appendix

Appendix A: Software and Hardware Requirements

Development: Python 3.11 (3.14 also verified), Node.js 20+, npm, Docker Desktop with Compose (optional), Visual Studio Code. Runtime: FastAPI 0.141, SQLAlchemy 2.0, Alembic, PyJWT, bcrypt, ReportLab, openpyxl; PostgreSQL 15 container or SQLite file. Clients: modern browser for the React dashboard; Expo Go or a development build for the mobile app. Test tooling: pytest, pytest-cov, Ruff, Vitest, Jest with jest-expo. Any machine with 8 GB RAM and 10 GB disk runs the full stack; GPU is not required (inference defaults to CPU).

Appendix B: Project Structure

backend/ app (main, Models, database, api, services, ai_engine, gateway, integrations), ml_models, scripts, tests, Dockerfile
web/ src (pages, components: auth, landing, layout, ui; utils), public, Dockerfile, nginx.conf
mobile/ src (screens, components, hooks, sync, notifications, context, theme), assets, App.js
migrations/ Alembic env and 11 version files (head f2b1d8e4a903)
root docker-compose.yml, docker-compose.prod.yml, .github/workflows/ci.yml, dev.ps1, run_all.ps1, CerebroKognit.bat, Markdown Files/ (SRS, PRD, guides)

Appendix C: Database Information

PostgreSQL 15 (Docker) or SQLite; SQLAlchemy 2.0 ORM; Alembic migrations. Tables (14): roles, users, user_roles, alarms, challenges, challenge_attempts, habit_scores, push_tokens, feedback, coach_assignments, auth_tokens, productivity_logs, audit_logs, announcements. Security-sensitive columns: expected_answers / answer_meta (server-only JSON), token_hash (SHA-256), expires_at, consumed_at, revoked.

Appendix D: Application Modules

Backend routers: auth, users, alarms, challenges, habits, sync, dashboard, analytics, notifications, feedback, productivity, announcements. Services: users, tokens, OAuth, challenges, verification, alarms, habit scoring, analytics, adaptive difficulty, recommendations, notifications, reports, admin, coach, audit, bootstrap. AI engine: difficulty model, feature rules, UCB1 bandit, anomaly detection, performance forecast. Web pages: Landing, Auth, Home, Dashboard, Alarms, Challenges, Reports, Profile, Settings, Admin. Mobile: Home/Alarms/Insights/Profile tabs, login, alarm form, challenge, settings, feedback screens, offline store, sync engine, notification scheduler.

Appendix E: Source Code Repository

https://github.com/PawanSimha/CerebroKognit-InfosysInternship-B3 (branch: main). Documentation set: README.md and Markdown Files/ (SRS, PRD, API reference, deployment, operations and user guides, test status).